Supplementary Materials

Companion to "Triplet architecture enables deep error-minimization in the genetic code" Submitted to the Journal of Molecular Evolution.

Table S1 — Sensitivity of Condition C to representative-set choice

Condition C (10-class triplet code at n = 3) was re-evaluated using three strategies for selecting one representative amino acid per RAA10 class: closest to centroid (used in the main text), farthest from centroid, and random class member. For the random strategy, three independent seeds (1, 2, 42) were run and the reported value is the mean across seeds ± sample standard deviation. Each entry was computed against its own 100 000-shuffle null distribution.

Representative set	Distortion z (D)	Dirichlet z (E)	D–E correlation
Closest to centroid (main text)	$-$14.79	$-$13.34	0.958
Farthest from centroid	$-$14.61	$-$13.18	0.965
Random class member (mean ± SD, 3 seeds)	$-$15.17 ± 0.16	$-$14.09 ± 0.54	0.961 ± 0.008

Per-seed values for the random strategy:

- D z-scores: seed 42 = $-$15.09, seed 1 = $-$15.36, seed 2 = $-$15.07
- E z-scores: seed 42 = $-$13.74, seed 1 = $-$14.71, seed 2 = $-$13.83
- D–E correlations: seed 42 = 0.955, seed 1 = 0.969, seed 2 = 0.958

Z-score ranges across strategies (using the random-strategy mean):

- Distortion z: **0.56**
- Dirichlet z: **0.92**

Both ranges are small compared to the ~15 SD displacement of the SGC from its null under both metrics, so the qualitative conclusion (SGC lies far outside the null under any reasonable choice of representative set) does not depend on the representative-selection rule.

Sources: `results/publication_controls.json` (closest, farthest, random seed 42); `results/sensitivity_extra` (random seeds 1 and 2); aggregation via `build_table_s1.py`.

Table S2 — Physicochemical descriptor set used for PCA

The amino acid property library begins with 22 physicochemical descriptors and retains 20 after correlation pruning at $|r| \geq 0.95$. The retained descriptors are then standardized (z-scored across the 20 amino acids) and compressed by PCA; the first 8 principal components explain 97.1 % of total variance and are used as the property vector $f(\cdot)$ throughout.

Retained descriptors (20):

1. `hydro_kd_z` — Kyte–Doolittle hydropathy

2. `vol_zam_z` — Zamyatnin side-chain volume
3. `polarity_gr_z` — Grantham polarity
4. `pI_z` — isoelectric point
5. `sc_pka_z` — side-chain pKa
6. `charge_pH7_z` — net side-chain charge at pH 7
7. `donors_z` — hydrogen-bond donor count
8. `acceptors_z` — hydrogen-bond acceptor count
9. `arom_z` — aromaticity indicator (0/1)
10. `ring_count_z` — number of rings
11. `rotb_z` — rotatable bond count
12. `bulkiness_z` — Zimmerman bulkiness
13. `cf_alpha_z` — Chou–Fasman α-helix propensity
14. `cf_beta_z` — Chou–Fasman β-sheet propensity
15. `cf_turn_z` — Chou–Fasman turn propensity
16. `acc_pref_z` — relative solvent accessibility
17. `flex_z` — normalized flexibility index
18. `helical_moment_z` — helical moment proxy
19. `beta_moment_z` — β-sheet moment proxy
20. `sc_mass_z` — side-chain mass

Dropped during correlation pruning (2 descriptors):

- `refractivity` ($|r| \geq 0.95$ with bulkiness)
- `hphob_fr` ($|r| \geq 0.95$ with hydro_kd)

PCA summary:

- Retained descriptors: 20
- Principal components retained: 8
- Cumulative variance explained: 0.9710

Sources: `src/io/aa_props_lib.py` (descriptor definitions); `build_aa_props.py` (pruning + PCA pipeline); `data/processed/aa_props_meta.json` (variance breakdown).

Figure S1 — Raw null distributions of noise distortion D for all three conditions

Figure file: `figS1_raw_distributions.png` (PNG, 300 DPI) and `figS1_raw_distributions.pdf` (vector, editable text).

Caption. Raw null distributions of noise distortion D for the three conditions used in the factorial analysis. Each null was generated by 10^6 degeneracy-preserving shuffles of amino-acid (or RAA10 class) labels across the sense codons, preserving per-class codon counts, fixing stops, and pinning AUG $\rightarrow$ Met. (A) Condition A: 10-class doublet (n = 2) code obtained by majority-vote projection of the SGC under the RAA10 alphabet; the SGC projection's observed value (solid red line) lies within the left tail of the null (z = $-$3.8, p = 8.3×10^{-5}, CV = 3.4 %). (B) Condition C: 10-class triplet (n = 3) code with RAA10 applied to the sense codons; the observed value (solid red line) lies outside the null (z = $-$14.8, $p < 10^{\{-\}\$\$}\{6\}\$$, CV = 2.1 %). (C) Condition B: the 20-AA SGC at n = 3; the observed value lies outside the null (z = $-$16.6, $p < 10^{\{-\}\$\$}\{6\}, CV = 1.6\hat{}\{6\}\$$ null samples; the solid red line marks the SGC's observed value. In (B) and (C) the red line sits well to the left of the dashed grey line — no random code in 10^6 samples reaches the SGC's noise distortion.

CV values are verified against `results/publication_controls.json` (A = 0.0343, C = 0.0211, B = 0.0164).

File manifest (supplementary files submitted alongside the manuscript)

Tables (raw data and typeset versions):

- `supplementary/table_S1.csv` — Table S1 raw values (closest, farthest, random-seeds mean ± SD)
- `supplementary/table_S1.tex` — Table S1 in LaTeX booktabs format
- `supplementary/table_S1_summary.json` — machine-readable numeric summary including per-seed values

Figures:

- `figures/figS1_raw_distributions.png` — Figure S1 at 300 DPI
- `figures/figS1_raw_distributions.pdf` — Figure S1 vector version with editable text (fonttype 42)

Supporting data (optional; can be provided on request or via a repository):

- `results/publication_controls.json` — full factorial + sensitivity numerical output (closest, farthest, random seed 42)
- `results/sensitivity_extra_seeds.json` — random seeds 1 and 2 used for the 3-seed mean in Table S1
- `results/null_distributions.npz` — raw 10^6 null distributions for all conditions (~46 MB, not typically included with submission; regeneratable via `python run_publication_controls.py --n-null 1000000 --workers 32`)

Scripts (reproducibility, in the companion code repository):

- `run_publication_controls.py` — runs the 2 × 2 factorial + sensitivity
- `run_sensitivity_extra_seeds.py` — runs the additional random seeds for Table S1
- `build_table_s1.py` — aggregates the three random seeds into Table S1